

Component-I: LSTM Sequence Generation

```
# =====
# Component-I: LSTM Sequence Generation
# =====

import numpy as np
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# -----
# Task 1: Load Dataset
# -----
data = """machine learning models learn patterns from data.
sequence models process data step by step.
recurrent neural networks are designed for sequential tasks.
rnn models maintain hidden states across time steps.

long short term memory networks solve long dependency problems.
lstm uses gates to control information flow.
gru models simplify the lstm architecture.
sequence prediction is useful in many applications.

language modeling predicts the next word in a sentence.
speech recognition processes audio sequences.
time series forecasting predicts future values.
music generation creates new melodies.

generative models learn probability distributions.
they generate new samples similar to training data.
sequence generation is widely used in artificial intelligence.
deep learning improves sequence modeling performance.
"""

# -----
# Task 2: Tokenization
# -----
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])
total_words = len(tokenizer.word_index) + 1

# -----
# Task 3: Create Sequences
# -----
input_sequences = []

for line in data.split('\n'):
    token_list = tokenizer.texts_to_sequences([line])[0]
```

```

        for i in range(1, len(token_list)):
            input_sequences.append(token_list[:i+1])

# Padding
max_len = max(len(seq) for seq in input_sequences)
input_sequences = np.array(
    pad_sequences(input_sequences, maxlen=max_len, padding='pre')
)

# Split into X and y
X = input_sequences[:, :-1]
y = input_sequences[:, -1]

# -----
# Task 4: Build LSTM Model
# -----
model = Sequential([
    Embedding(total_words, 100, input_length=max_len-1),
    LSTM(150),
    Dense(total_words, activation='softmax')
])

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam')

# -----
# Task 5: Train Model
# -----
model.fit(X, y, epochs=200, verbose=1)

# -----
# Task 6: Generate Text
# -----
def generate_text(seed_text, next_words):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences(
            [token_list], maxlen=max_len-1, padding='pre'
        )

        predicted = np.argmax(model.predict(token_list), axis=-1)

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text

# Example Output

```

```
print("\nGenerated Text (LSTM):")
print(generate_text("machine learning", 6))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
embedding.py:100: UserWarning: Argument `input_length` is deprecated.
Just remove it.
```

```
warnings.warn(
```

```
Epoch 1/200
3/3 _____ 4s 22ms/step - loss: 4.4649
Epoch 2/200
3/3 _____ 0s 19ms/step - loss: 4.4478
Epoch 3/200
3/3 _____ 0s 19ms/step - loss: 4.4278
Epoch 4/200
3/3 _____ 0s 22ms/step - loss: 4.3976
Epoch 5/200
3/3 _____ 0s 24ms/step - loss: 4.3403
Epoch 6/200
3/3 _____ 0s 21ms/step - loss: 4.2569
Epoch 7/200
3/3 _____ 0s 21ms/step - loss: 4.2336
Epoch 8/200
3/3 _____ 0s 21ms/step - loss: 4.2054
Epoch 9/200
3/3 _____ 0s 20ms/step - loss: 4.1581
Epoch 10/200
3/3 _____ 0s 21ms/step - loss: 4.1393
Epoch 11/200
3/3 _____ 0s 31ms/step - loss: 4.1052
Epoch 12/200
3/3 _____ 0s 22ms/step - loss: 4.0733
Epoch 13/200
3/3 _____ 0s 22ms/step - loss: 4.0371
Epoch 14/200
3/3 _____ 0s 23ms/step - loss: 3.9856
Epoch 15/200
3/3 _____ 0s 19ms/step - loss: 3.9457
Epoch 16/200
3/3 _____ 0s 19ms/step - loss: 3.8852
Epoch 17/200
3/3 _____ 0s 22ms/step - loss: 3.8243
Epoch 18/200
3/3 _____ 0s 20ms/step - loss: 3.7541
Epoch 19/200
3/3 _____ 0s 21ms/step - loss: 3.6780
Epoch 20/200
3/3 _____ 0s 21ms/step - loss: 3.6155
Epoch 21/200
3/3 _____ 0s 22ms/step - loss: 3.5494
```

```
Epoch 22/200
3/3 _____ 0s 22ms/step - loss: 3.4843
Epoch 23/200
3/3 _____ 0s 23ms/step - loss: 3.3670
Epoch 24/200
3/3 _____ 0s 27ms/step - loss: 3.3111
Epoch 25/200
3/3 _____ 0s 29ms/step - loss: 3.1964
Epoch 26/200
3/3 _____ 0s 20ms/step - loss: 3.1214
Epoch 27/200
3/3 _____ 0s 15ms/step - loss: 3.0237
Epoch 28/200
3/3 _____ 0s 15ms/step - loss: 2.9225
Epoch 29/200
3/3 _____ 0s 15ms/step - loss: 2.8101
Epoch 30/200
3/3 _____ 0s 15ms/step - loss: 2.7277
Epoch 31/200
3/3 _____ 0s 15ms/step - loss: 2.6233
Epoch 32/200
3/3 _____ 0s 15ms/step - loss: 2.5109
Epoch 33/200
3/3 _____ 0s 15ms/step - loss: 2.3983
Epoch 34/200
3/3 _____ 0s 15ms/step - loss: 2.3006
Epoch 35/200
3/3 _____ 0s 16ms/step - loss: 2.2027
Epoch 36/200
3/3 _____ 0s 15ms/step - loss: 2.1158
Epoch 37/200
3/3 _____ 0s 15ms/step - loss: 2.0186
Epoch 38/200
3/3 _____ 0s 15ms/step - loss: 1.9349
Epoch 39/200
3/3 _____ 0s 15ms/step - loss: 1.8627
Epoch 40/200
3/3 _____ 0s 15ms/step - loss: 1.7772
Epoch 41/200
3/3 _____ 0s 15ms/step - loss: 1.6950
Epoch 42/200
3/3 _____ 0s 22ms/step - loss: 1.6394
Epoch 43/200
3/3 _____ 0s 15ms/step - loss: 1.5641
Epoch 44/200
3/3 _____ 0s 15ms/step - loss: 1.5031
Epoch 45/200
3/3 _____ 0s 15ms/step - loss: 1.4517
Epoch 46/200
```

3/3	_____	0s 23ms/step	- loss: 1.3862
Epoch 47/200			
3/3	_____	0s 44ms/step	- loss: 1.3421
Epoch 48/200			
3/3	_____	0s 37ms/step	- loss: 1.2828
Epoch 49/200			
3/3	_____	0s 46ms/step	- loss: 1.2422
Epoch 50/200			
3/3	_____	0s 32ms/step	- loss: 1.1912
Epoch 51/200			
3/3	_____	0s 37ms/step	- loss: 1.1454
Epoch 52/200			
3/3	_____	0s 38ms/step	- loss: 1.1111
Epoch 53/200			
3/3	_____	0s 16ms/step	- loss: 1.0726
Epoch 54/200			
3/3	_____	0s 16ms/step	- loss: 1.0393
Epoch 55/200			
3/3	_____	0s 16ms/step	- loss: 1.0046
Epoch 56/200			
3/3	_____	0s 15ms/step	- loss: 0.9742
Epoch 57/200			
3/3	_____	0s 17ms/step	- loss: 0.9350
Epoch 58/200			
3/3	_____	0s 15ms/step	- loss: 0.9060
Epoch 59/200			
3/3	_____	0s 14ms/step	- loss: 0.8771
Epoch 60/200			
3/3	_____	0s 15ms/step	- loss: 0.8421
Epoch 61/200			
3/3	_____	0s 15ms/step	- loss: 0.8236
Epoch 62/200			
3/3	_____	0s 17ms/step	- loss: 0.7919
Epoch 63/200			
3/3	_____	0s 15ms/step	- loss: 0.7697
Epoch 64/200			
3/3	_____	0s 16ms/step	- loss: 0.7484
Epoch 65/200			
3/3	_____	0s 15ms/step	- loss: 0.7254
Epoch 66/200			
3/3	_____	0s 16ms/step	- loss: 0.6996
Epoch 67/200			
3/3	_____	0s 15ms/step	- loss: 0.6841
Epoch 68/200			
3/3	_____	0s 16ms/step	- loss: 0.6628
Epoch 69/200			
3/3	_____	0s 16ms/step	- loss: 0.6424
Epoch 70/200			
3/3	_____	0s 15ms/step	- loss: 0.6283

Epoch 71/200			
3/3	_____	0s 16ms/step	loss: 0.6054
Epoch 72/200			
3/3	_____	0s 15ms/step	loss: 0.5898
Epoch 73/200			
3/3	_____	0s 16ms/step	loss: 0.5715
Epoch 74/200			
3/3	_____	0s 16ms/step	loss: 0.5595
Epoch 75/200			
3/3	_____	0s 15ms/step	loss: 0.5412
Epoch 76/200			
3/3	_____	0s 16ms/step	loss: 0.5263
Epoch 77/200			
3/3	_____	0s 15ms/step	loss: 0.5112
Epoch 78/200			
3/3	_____	0s 15ms/step	loss: 0.4972
Epoch 79/200			
3/3	_____	0s 16ms/step	loss: 0.4842
Epoch 80/200			
3/3	_____	0s 16ms/step	loss: 0.4696
Epoch 81/200			
3/3	_____	0s 16ms/step	loss: 0.4583
Epoch 82/200			
3/3	_____	0s 18ms/step	loss: 0.4454
Epoch 83/200			
3/3	_____	0s 16ms/step	loss: 0.4329
Epoch 84/200			
3/3	_____	0s 15ms/step	loss: 0.4203
Epoch 85/200			
3/3	_____	0s 15ms/step	loss: 0.4130
Epoch 86/200			
3/3	_____	0s 16ms/step	loss: 0.4049
Epoch 87/200			
3/3	_____	0s 16ms/step	loss: 0.3910
Epoch 88/200			
3/3	_____	0s 15ms/step	loss: 0.3803
Epoch 89/200			
3/3	_____	0s 17ms/step	loss: 0.3718
Epoch 90/200			
3/3	_____	0s 15ms/step	loss: 0.3641
Epoch 91/200			
3/3	_____	0s 15ms/step	loss: 0.3500
Epoch 92/200			
3/3	_____	0s 15ms/step	loss: 0.3451
Epoch 93/200			
3/3	_____	0s 16ms/step	loss: 0.3345
Epoch 94/200			
3/3	_____	0s 16ms/step	loss: 0.3286
Epoch 95/200			

3/3	_____	0s 15ms/step	- loss: 0.3179
Epoch 96/200			
3/3	_____	0s 15ms/step	- loss: 0.3115
Epoch 97/200			
3/3	_____	0s 15ms/step	- loss: 0.3049
Epoch 98/200			
3/3	_____	0s 21ms/step	- loss: 0.2949
Epoch 99/200			
3/3	_____	0s 15ms/step	- loss: 0.2903
Epoch 100/200			
3/3	_____	0s 15ms/step	- loss: 0.2812
Epoch 101/200			
3/3	_____	0s 16ms/step	- loss: 0.2739
Epoch 102/200			
3/3	_____	0s 15ms/step	- loss: 0.2683
Epoch 103/200			
3/3	_____	0s 16ms/step	- loss: 0.2623
Epoch 104/200			
3/3	_____	0s 15ms/step	- loss: 0.2541
Epoch 105/200			
3/3	_____	0s 16ms/step	- loss: 0.2501
Epoch 106/200			
3/3	_____	0s 17ms/step	- loss: 0.2446
Epoch 107/200			
3/3	_____	0s 15ms/step	- loss: 0.2397
Epoch 108/200			
3/3	_____	0s 16ms/step	- loss: 0.2320
Epoch 109/200			
3/3	_____	0s 16ms/step	- loss: 0.2284
Epoch 110/200			
3/3	_____	0s 15ms/step	- loss: 0.2247
Epoch 111/200			
3/3	_____	0s 15ms/step	- loss: 0.2171
Epoch 112/200			
3/3	_____	0s 16ms/step	- loss: 0.2125
Epoch 113/200			
3/3	_____	0s 15ms/step	- loss: 0.2083
Epoch 114/200			
3/3	_____	0s 16ms/step	- loss: 0.2045
Epoch 115/200			
3/3	_____	0s 22ms/step	- loss: 0.2001
Epoch 116/200			
3/3	_____	0s 16ms/step	- loss: 0.1953
Epoch 117/200			
3/3	_____	0s 15ms/step	- loss: 0.1912
Epoch 118/200			
3/3	_____	0s 15ms/step	- loss: 0.1876
Epoch 119/200			
3/3	_____	0s 17ms/step	- loss: 0.1842

```
Epoch 120/200
3/3 _____ 0s 16ms/step - loss: 0.1796
Epoch 121/200
3/3 _____ 0s 16ms/step - loss: 0.1764
Epoch 122/200
3/3 _____ 0s 15ms/step - loss: 0.1737
Epoch 123/200
3/3 _____ 0s 15ms/step - loss: 0.1696
Epoch 124/200
3/3 _____ 0s 16ms/step - loss: 0.1664
Epoch 125/200
3/3 _____ 0s 15ms/step - loss: 0.1642
Epoch 126/200
3/3 _____ 0s 15ms/step - loss: 0.1620
Epoch 127/200
3/3 _____ 0s 16ms/step - loss: 0.1591
Epoch 128/200
3/3 _____ 0s 17ms/step - loss: 0.1554
Epoch 129/200
3/3 _____ 0s 15ms/step - loss: 0.1525
Epoch 130/200
3/3 _____ 0s 16ms/step - loss: 0.1497
Epoch 131/200
3/3 _____ 0s 16ms/step - loss: 0.1480
Epoch 132/200
3/3 _____ 0s 25ms/step - loss: 0.1454
Epoch 133/200
3/3 _____ 0s 16ms/step - loss: 0.1431
Epoch 134/200
3/3 _____ 0s 16ms/step - loss: 0.1408
Epoch 135/200
3/3 _____ 0s 15ms/step - loss: 0.1394
Epoch 136/200
3/3 _____ 0s 15ms/step - loss: 0.1371
Epoch 137/200
3/3 _____ 0s 15ms/step - loss: 0.1344
Epoch 138/200
3/3 _____ 0s 15ms/step - loss: 0.1328
Epoch 139/200
3/3 _____ 0s 15ms/step - loss: 0.1304
Epoch 140/200
3/3 _____ 0s 19ms/step - loss: 0.1285
Epoch 141/200
3/3 _____ 0s 16ms/step - loss: 0.1267
Epoch 142/200
3/3 _____ 0s 15ms/step - loss: 0.1248
Epoch 143/200
3/3 _____ 0s 15ms/step - loss: 0.1227
Epoch 144/200
```



```
3/3 _____ 0s 15ms/step - loss: 0.1219
Epoch 145/200
3/3 _____ 0s 16ms/step - loss: 0.1205
Epoch 146/200
3/3 _____ 0s 16ms/step - loss: 0.1195
Epoch 147/200
3/3 _____ 0s 16ms/step - loss: 0.1162
Epoch 148/200
3/3 _____ 0s 16ms/step - loss: 0.1154
Epoch 149/200
3/3 _____ 0s 15ms/step - loss: 0.1140
Epoch 150/200
3/3 _____ 0s 15ms/step - loss: 0.1128
Epoch 151/200
3/3 _____ 0s 16ms/step - loss: 0.1107
Epoch 152/200
3/3 _____ 0s 16ms/step - loss: 0.1097
Epoch 153/200
3/3 _____ 0s 24ms/step - loss: 0.1082
Epoch 154/200
3/3 _____ 0s 23ms/step - loss: 0.1070
Epoch 155/200
3/3 _____ 0s 24ms/step - loss: 0.1063
Epoch 156/200
3/3 _____ 0s 22ms/step - loss: 0.1051
Epoch 157/200
3/3 _____ 0s 21ms/step - loss: 0.1034
Epoch 158/200
3/3 _____ 0s 21ms/step - loss: 0.1029
Epoch 159/200
3/3 _____ 0s 22ms/step - loss: 0.1013
Epoch 160/200
3/3 _____ 0s 21ms/step - loss: 0.1003
Epoch 161/200
3/3 _____ 0s 21ms/step - loss: 0.0995
Epoch 162/200
3/3 _____ 0s 25ms/step - loss: 0.0978
Epoch 163/200
3/3 _____ 0s 22ms/step - loss: 0.0969
Epoch 164/200
3/3 _____ 0s 22ms/step - loss: 0.0955
Epoch 165/200
3/3 _____ 0s 21ms/step - loss: 0.0951
Epoch 166/200
3/3 _____ 0s 22ms/step - loss: 0.0945
Epoch 167/200
3/3 _____ 0s 21ms/step - loss: 0.0932
Epoch 168/200
3/3 _____ 0s 24ms/step - loss: 0.0926
```

```
Epoch 169/200
3/3 _____ 0s 25ms/step - loss: 0.0913
Epoch 170/200
3/3 _____ 0s 22ms/step - loss: 0.0906
Epoch 171/200
3/3 _____ 0s 25ms/step - loss: 0.0898
Epoch 172/200
3/3 _____ 0s 24ms/step - loss: 0.0891
Epoch 173/200
3/3 _____ 0s 24ms/step - loss: 0.0880
Epoch 174/200
3/3 _____ 0s 29ms/step - loss: 0.0873
Epoch 175/200
3/3 _____ 0s 22ms/step - loss: 0.0869
Epoch 176/200
3/3 _____ 0s 18ms/step - loss: 0.0863
Epoch 177/200
3/3 _____ 0s 15ms/step - loss: 0.0857
Epoch 178/200
3/3 _____ 0s 16ms/step - loss: 0.0846
Epoch 179/200
3/3 _____ 0s 17ms/step - loss: 0.0840
Epoch 180/200
3/3 _____ 0s 20ms/step - loss: 0.0827
Epoch 181/200
3/3 _____ 0s 16ms/step - loss: 0.0825
Epoch 182/200
3/3 _____ 0s 17ms/step - loss: 0.0814
Epoch 183/200
3/3 _____ 0s 15ms/step - loss: 0.0814
Epoch 184/200
3/3 _____ 0s 18ms/step - loss: 0.0813
Epoch 185/200
3/3 _____ 0s 15ms/step - loss: 0.0808
Epoch 186/200
3/3 _____ 0s 15ms/step - loss: 0.0798
Epoch 187/200
3/3 _____ 0s 16ms/step - loss: 0.0792
Epoch 188/200
3/3 _____ 0s 15ms/step - loss: 0.0789
Epoch 189/200
3/3 _____ 0s 15ms/step - loss: 0.0780
Epoch 190/200
3/3 _____ 0s 16ms/step - loss: 0.0777
Epoch 191/200
3/3 _____ 0s 16ms/step - loss: 0.0767
Epoch 192/200
3/3 _____ 0s 15ms/step - loss: 0.0769
Epoch 193/200
```

```
3/3 _____ 0s 16ms/step - loss: 0.0760
Epoch 194/200
3/3 _____ 0s 16ms/step - loss: 0.0758
Epoch 195/200
3/3 _____ 0s 25ms/step - loss: 0.0745
Epoch 196/200
3/3 _____ 0s 16ms/step - loss: 0.0748
Epoch 197/200
3/3 _____ 0s 16ms/step - loss: 0.0732
Epoch 198/200
3/3 _____ 0s 15ms/step - loss: 0.0740
Epoch 199/200
3/3 _____ 0s 17ms/step - loss: 0.0735
Epoch 200/200
3/3 _____ 0s 15ms/step - loss: 0.0726
```

Generated Text (LSTM):

```
1/1 _____ 0s 186ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 39ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 43ms/step
1/1 _____ 0s 40ms/step
```

machine learning models learn patterns from data data

Example Output

```
print("\nGenerated Text (LSTM):")
print(generate_text("machine learning", 6))
```

Generated Text (LSTM):

```
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
```

machine learning models learn patterns from data step

Example Output

```
print("\nGenerated Text (LSTM):")
print(generate_text("long short term", 6))
```

Generated Text (LSTM):

```
1/1 _____ 0s 36ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 36ms/step
1/1 _____ 0s 35ms/step
```

1/1 ————— 0s 35ms/step
long short term memory networks solve long dependency problems

Component-II: Transformer Sequence Generation

```
# =====  
# Component-II: Transformer Sequence Generation  
# =====  
  
import tensorflow as tf  
from tensorflow.keras.layers import Embedding, Dense,  
LayerNormalization, Dropout  
from tensorflow.keras.models import Model  
import numpy as np  
  
# -----  
# Task 1 & 2: Dataset + Tokenization  
# -----  
tokenizer = Tokenizer()  
tokenizer.fit_on_texts([data])  
total_words = len(tokenizer.word_index) + 1  
  
input_sequences = []  
for line in data.split('\n'):  
    token_list = tokenizer.texts_to_sequences([line])[0]  
    for i in range(1, len(token_list)):  
        input_sequences.append(token_list[:i+1])  
  
max_len = max(len(seq) for seq in input_sequences)  
input_sequences = pad_sequences(input_sequences, maxlen=max_len,  
padding='pre')  
  
X = input_sequences[:, :-1]  
y = input_sequences[:, -1]  
  
# -----  
# Task 3: Positional Encoding  
# -----  
class PositionalEncoding(tf.keras.layers.Layer):  
    def __init__(self, max_len, d_model):  
        super().__init__()  
        self.pos_encoding = self.positional_encoding(max_len, d_model)  
  
    def positional_encoding(self, position, d_model):  
        angles = np.arange(position)[:, np.newaxis] / np.power(  
            10000, (2 * (np.arange(d_model)[np.newaxis, :] // 2)) /  
d_model  
        )  
        pos_encoding = angles.copy()  
        pos_encoding[:, 0::2] = np.sin(angles[:, 0::2])
```

```

        pos_encoding[:, 1::2] = np.cos(angles[:, 1::2])
        return tf.cast(pos_encoding[np.newaxis, ...],
dtype=tf.float32)

    def call(self, x):
        return x + self.pos_encoding[:, :tf.shape(x)[1], :]

# -----
# Task 4: Transformer Block
# -----
class TransformerBlock(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim):
        super().__init__()
        self.att =
tf.keras.layers.MultiHeadAttention(num_heads=num_heads,
key_dim=embed_dim)
        self.ffn = tf.keras.Sequential([
            Dense(ff_dim, activation='relu'),
            Dense(embed_dim),
        ])
        self.norm1 = LayerNormalization()
        self.norm2 = LayerNormalization()
        self.dropout1 = Dropout(0.1)
        self.dropout2 = Dropout(0.1)

    def call(self, inputs):
        attn_output = self.att(inputs, inputs)
        out1 = self.norm1(inputs + self.dropout1(attn_output))
        ffn_output = self.ffn(out1)
        return self.norm2(out1 + self.dropout2(ffn_output))

# -----
# Build Transformer Model
# -----
embed_dim = 64
num_heads = 2
ff_dim = 128

inputs = tf.keras.Input(shape=(max_len-1,))
x = Embedding(total_words, embed_dim)(inputs)
x = PositionalEncoding(max_len, embed_dim)(x)
x = TransformerBlock(embed_dim, num_heads, ff_dim)(x)
x = tf.keras.layers.GlobalAveragePooling1D()(x)
outputs = Dense(total_words, activation='softmax')(x)

model = Model(inputs, outputs)

model.compile(loss='sparse_categorical_crossentropy',
optimizer='adam')

```

```

# -----
# Task 5: Train
# -----
model.fit(X, y, epochs=200, verbose=1)

# -----
# Task 6: Generate Text
# -----
def generate_text_transformer(seed_text, next_words):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences(
            [token_list], maxlen=max_len-1, padding='pre'
        )

        predicted = np.argmax(model.predict(token_list), axis=-1)

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text

print("\nGenerated Text (Transformer):")
print(generate_text_transformer("deep learning", 6))

```

```

Epoch 1/200
3/3 _____ 10s 1s/step - loss: 4.7923
Epoch 2/200
3/3 _____ 0s 12ms/step - loss: 4.5582
Epoch 3/200
3/3 _____ 0s 13ms/step - loss: 4.4654
Epoch 4/200
3/3 _____ 0s 15ms/step - loss: 4.4076
Epoch 5/200
3/3 _____ 0s 14ms/step - loss: 4.3909
Epoch 6/200
3/3 _____ 0s 14ms/step - loss: 4.3504
Epoch 7/200
3/3 _____ 0s 13ms/step - loss: 4.3271
Epoch 8/200
3/3 _____ 0s 13ms/step - loss: 4.2974
Epoch 9/200
3/3 _____ 0s 13ms/step - loss: 4.2701
Epoch 10/200
3/3 _____ 0s 14ms/step - loss: 4.2583
Epoch 11/200
3/3 _____ 0s 14ms/step - loss: 4.2696
Epoch 12/200

```

3/3	_____	0s	13ms/step	-	loss: 4.2507
Epoch 13/200					
3/3	_____	0s	13ms/step	-	loss: 4.2422
Epoch 14/200					
3/3	_____	0s	13ms/step	-	loss: 4.2116
Epoch 15/200					
3/3	_____	0s	13ms/step	-	loss: 4.2142
Epoch 16/200					
3/3	_____	0s	14ms/step	-	loss: 4.2095
Epoch 17/200					
3/3	_____	0s	13ms/step	-	loss: 4.1967
Epoch 18/200					
3/3	_____	0s	18ms/step	-	loss: 4.1682
Epoch 19/200					
3/3	_____	0s	13ms/step	-	loss: 4.1548
Epoch 20/200					
3/3	_____	0s	15ms/step	-	loss: 4.1387
Epoch 21/200					
3/3	_____	0s	14ms/step	-	loss: 4.1176
Epoch 22/200					
3/3	_____	0s	14ms/step	-	loss: 4.0932
Epoch 23/200					
3/3	_____	0s	13ms/step	-	loss: 4.0672
Epoch 24/200					
3/3	_____	0s	15ms/step	-	loss: 4.0266
Epoch 25/200					
3/3	_____	0s	13ms/step	-	loss: 3.9868
Epoch 26/200					
3/3	_____	0s	15ms/step	-	loss: 3.9372
Epoch 27/200					
3/3	_____	0s	13ms/step	-	loss: 3.8986
Epoch 28/200					
3/3	_____	0s	13ms/step	-	loss: 3.8311
Epoch 29/200					
3/3	_____	0s	13ms/step	-	loss: 3.7657
Epoch 30/200					
3/3	_____	0s	14ms/step	-	loss: 3.6908
Epoch 31/200					
3/3	_____	0s	13ms/step	-	loss: 3.6268
Epoch 32/200					
3/3	_____	0s	13ms/step	-	loss: 3.5572
Epoch 33/200					
3/3	_____	0s	13ms/step	-	loss: 3.4470
Epoch 34/200					
3/3	_____	0s	13ms/step	-	loss: 3.3602
Epoch 35/200					
3/3	_____	0s	13ms/step	-	loss: 3.3087
Epoch 36/200					
3/3	_____	0s	13ms/step	-	loss: 3.1869

```
Epoch 37/200
3/3 _____ 0s 18ms/step - loss: 3.1248
Epoch 38/200
3/3 _____ 0s 13ms/step - loss: 2.9938
Epoch 39/200
3/3 _____ 0s 13ms/step - loss: 2.9344
Epoch 40/200
3/3 _____ 0s 13ms/step - loss: 2.8469
Epoch 41/200
3/3 _____ 0s 13ms/step - loss: 2.7274
Epoch 42/200
3/3 _____ 0s 13ms/step - loss: 2.6061
Epoch 43/200
3/3 _____ 0s 13ms/step - loss: 2.5327
Epoch 44/200
3/3 _____ 0s 13ms/step - loss: 2.4455
Epoch 45/200
3/3 _____ 0s 13ms/step - loss: 2.3636
Epoch 46/200
3/3 _____ 0s 13ms/step - loss: 2.2225
Epoch 47/200
3/3 _____ 0s 13ms/step - loss: 2.1622
Epoch 48/200
3/3 _____ 0s 14ms/step - loss: 2.0420
Epoch 49/200
3/3 _____ 0s 13ms/step - loss: 1.9816
Epoch 50/200
3/3 _____ 0s 13ms/step - loss: 1.8884
Epoch 51/200
3/3 _____ 0s 13ms/step - loss: 1.8035
Epoch 52/200
3/3 _____ 0s 13ms/step - loss: 1.7145
Epoch 53/200
3/3 _____ 0s 14ms/step - loss: 1.6031
Epoch 54/200
3/3 _____ 0s 14ms/step - loss: 1.5777
Epoch 55/200
3/3 _____ 0s 14ms/step - loss: 1.4748
Epoch 56/200
3/3 _____ 0s 17ms/step - loss: 1.4118
Epoch 57/200
3/3 _____ 0s 14ms/step - loss: 1.3515
Epoch 58/200
3/3 _____ 0s 13ms/step - loss: 1.2971
Epoch 59/200
3/3 _____ 0s 13ms/step - loss: 1.2493
Epoch 60/200
3/3 _____ 0s 13ms/step - loss: 1.2014
Epoch 61/200
```



```
3/3 _____ 0s 13ms/step - loss: 1.1080
Epoch 62/200
3/3 _____ 0s 13ms/step - loss: 1.0804
Epoch 63/200
3/3 _____ 0s 13ms/step - loss: 1.0380
Epoch 64/200
3/3 _____ 0s 22ms/step - loss: 0.9721
Epoch 65/200
3/3 _____ 0s 19ms/step - loss: 0.9152
Epoch 66/200
3/3 _____ 0s 21ms/step - loss: 0.8993
Epoch 67/200
3/3 _____ 0s 18ms/step - loss: 0.8453
Epoch 68/200
3/3 _____ 0s 17ms/step - loss: 0.8325
Epoch 69/200
3/3 _____ 0s 17ms/step - loss: 0.7980
Epoch 70/200
3/3 _____ 0s 28ms/step - loss: 0.7513
Epoch 71/200
3/3 _____ 0s 18ms/step - loss: 0.7139
Epoch 72/200
3/3 _____ 0s 20ms/step - loss: 0.7021
Epoch 73/200
3/3 _____ 0s 18ms/step - loss: 0.6650
Epoch 74/200
3/3 _____ 0s 20ms/step - loss: 0.6423
Epoch 75/200
3/3 _____ 0s 21ms/step - loss: 0.6005
Epoch 76/200
3/3 _____ 0s 20ms/step - loss: 0.5802
Epoch 77/200
3/3 _____ 0s 20ms/step - loss: 0.5511
Epoch 78/200
3/3 _____ 0s 19ms/step - loss: 0.5279
Epoch 79/200
3/3 _____ 0s 25ms/step - loss: 0.5295
Epoch 80/200
3/3 _____ 0s 21ms/step - loss: 0.4873
Epoch 81/200
3/3 _____ 0s 18ms/step - loss: 0.4699
Epoch 82/200
3/3 _____ 0s 14ms/step - loss: 0.4646
Epoch 83/200
3/3 _____ 0s 13ms/step - loss: 0.4316
Epoch 84/200
3/3 _____ 0s 14ms/step - loss: 0.4194
Epoch 85/200
3/3 _____ 0s 14ms/step - loss: 0.3924
```

```
Epoch 86/200
3/3 _____ 0s 14ms/step - loss: 0.3817
Epoch 87/200
3/3 _____ 0s 14ms/step - loss: 0.3722
Epoch 88/200
3/3 _____ 0s 14ms/step - loss: 0.3658
Epoch 89/200
3/3 _____ 0s 13ms/step - loss: 0.3364
Epoch 90/200
3/3 _____ 0s 14ms/step - loss: 0.3300
Epoch 91/200
3/3 _____ 0s 13ms/step - loss: 0.3189
Epoch 92/200
3/3 _____ 0s 13ms/step - loss: 0.3170
Epoch 93/200
3/3 _____ 0s 13ms/step - loss: 0.3018
Epoch 94/200
3/3 _____ 0s 13ms/step - loss: 0.2955
Epoch 95/200
3/3 _____ 0s 15ms/step - loss: 0.2761
Epoch 96/200
3/3 _____ 0s 13ms/step - loss: 0.2655
Epoch 97/200
3/3 _____ 0s 13ms/step - loss: 0.2626
Epoch 98/200
3/3 _____ 0s 14ms/step - loss: 0.2488
Epoch 99/200
3/3 _____ 0s 14ms/step - loss: 0.2544
Epoch 100/200
3/3 _____ 0s 13ms/step - loss: 0.2324
Epoch 101/200
3/3 _____ 0s 13ms/step - loss: 0.2424
Epoch 102/200
3/3 _____ 0s 15ms/step - loss: 0.2268
Epoch 103/200
3/3 _____ 0s 13ms/step - loss: 0.2263
Epoch 104/200
3/3 _____ 0s 13ms/step - loss: 0.2145
Epoch 105/200
3/3 _____ 0s 13ms/step - loss: 0.2101
Epoch 106/200
3/3 _____ 0s 14ms/step - loss: 0.2154
Epoch 107/200
3/3 _____ 0s 13ms/step - loss: 0.2033
Epoch 108/200
3/3 _____ 0s 14ms/step - loss: 0.2050
Epoch 109/200
3/3 _____ 0s 14ms/step - loss: 0.2044
Epoch 110/200
```

```
3/3 _____ 0s 13ms/step - loss: 0.1925
Epoch 111/200
3/3 _____ 0s 13ms/step - loss: 0.1952
Epoch 112/200
3/3 _____ 0s 14ms/step - loss: 0.1814
Epoch 113/200
3/3 _____ 0s 13ms/step - loss: 0.1787
Epoch 114/200
3/3 _____ 0s 16ms/step - loss: 0.1680
Epoch 115/200
3/3 _____ 0s 14ms/step - loss: 0.1617
Epoch 116/200
3/3 _____ 0s 14ms/step - loss: 0.1605
Epoch 117/200
3/3 _____ 0s 17ms/step - loss: 0.1563
Epoch 118/200
3/3 _____ 0s 13ms/step - loss: 0.1510
Epoch 119/200
3/3 _____ 0s 14ms/step - loss: 0.1460
Epoch 120/200
3/3 _____ 0s 16ms/step - loss: 0.1456
Epoch 121/200
3/3 _____ 0s 13ms/step - loss: 0.1496
Epoch 122/200
3/3 _____ 0s 13ms/step - loss: 0.1376
Epoch 123/200
3/3 _____ 0s 13ms/step - loss: 0.1391
Epoch 124/200
3/3 _____ 0s 15ms/step - loss: 0.1380
Epoch 125/200
3/3 _____ 0s 13ms/step - loss: 0.1263
Epoch 126/200
3/3 _____ 0s 13ms/step - loss: 0.1312
Epoch 127/200
3/3 _____ 0s 13ms/step - loss: 0.1241
Epoch 128/200
3/3 _____ 0s 13ms/step - loss: 0.1252
Epoch 129/200
3/3 _____ 0s 13ms/step - loss: 0.1232
Epoch 130/200
3/3 _____ 0s 14ms/step - loss: 0.1236
Epoch 131/200
3/3 _____ 0s 14ms/step - loss: 0.1184
Epoch 132/200
3/3 _____ 0s 14ms/step - loss: 0.1172
Epoch 133/200
3/3 _____ 0s 14ms/step - loss: 0.1196
Epoch 134/200
3/3 _____ 0s 14ms/step - loss: 0.1092
```

```
Epoch 135/200
3/3 _____ 0s 13ms/step - loss: 0.1132
Epoch 136/200
3/3 _____ 0s 14ms/step - loss: 0.1067
Epoch 137/200
3/3 _____ 0s 14ms/step - loss: 0.1049
Epoch 138/200
3/3 _____ 0s 13ms/step - loss: 0.1114
Epoch 139/200
3/3 _____ 0s 13ms/step - loss: 0.1014
Epoch 140/200
3/3 _____ 0s 14ms/step - loss: 0.1078
Epoch 141/200
3/3 _____ 0s 14ms/step - loss: 0.1080
Epoch 142/200
3/3 _____ 0s 13ms/step - loss: 0.1014
Epoch 143/200
3/3 _____ 0s 13ms/step - loss: 0.1020
Epoch 144/200
3/3 _____ 0s 13ms/step - loss: 0.0927
Epoch 145/200
3/3 _____ 0s 14ms/step - loss: 0.1016
Epoch 146/200
3/3 _____ 0s 14ms/step - loss: 0.0973
Epoch 147/200
3/3 _____ 0s 19ms/step - loss: 0.0865
Epoch 148/200
3/3 _____ 0s 14ms/step - loss: 0.0916
Epoch 149/200
3/3 _____ 0s 14ms/step - loss: 0.1101
Epoch 150/200
3/3 _____ 0s 14ms/step - loss: 0.0965
Epoch 151/200
3/3 _____ 0s 15ms/step - loss: 0.0835
Epoch 152/200
3/3 _____ 0s 14ms/step - loss: 0.0911
Epoch 153/200
3/3 _____ 0s 13ms/step - loss: 0.1010
Epoch 154/200
3/3 _____ 0s 13ms/step - loss: 0.0857
Epoch 155/200
3/3 _____ 0s 14ms/step - loss: 0.0889
Epoch 156/200
3/3 _____ 0s 13ms/step - loss: 0.0870
Epoch 157/200
3/3 _____ 0s 14ms/step - loss: 0.0843
Epoch 158/200
3/3 _____ 0s 14ms/step - loss: 0.0829
Epoch 159/200
```

```
3/3 _____ 0s 14ms/step - loss: 0.0864
Epoch 160/200
3/3 _____ 0s 13ms/step - loss: 0.0864
Epoch 161/200
3/3 _____ 0s 14ms/step - loss: 0.0887
Epoch 162/200
3/3 _____ 0s 15ms/step - loss: 0.0827
Epoch 163/200
3/3 _____ 0s 14ms/step - loss: 0.0826
Epoch 164/200
3/3 _____ 0s 14ms/step - loss: 0.0810
Epoch 165/200
3/3 _____ 0s 15ms/step - loss: 0.0720
Epoch 166/200
3/3 _____ 0s 14ms/step - loss: 0.0773
Epoch 167/200
3/3 _____ 0s 14ms/step - loss: 0.0763
Epoch 168/200
3/3 _____ 0s 13ms/step - loss: 0.0742
Epoch 169/200
3/3 _____ 0s 14ms/step - loss: 0.0728
Epoch 170/200
3/3 _____ 0s 14ms/step - loss: 0.0814
Epoch 171/200
3/3 _____ 0s 14ms/step - loss: 0.0728
Epoch 172/200
3/3 _____ 0s 14ms/step - loss: 0.0733
Epoch 173/200
3/3 _____ 0s 14ms/step - loss: 0.0804
Epoch 174/200
3/3 _____ 0s 15ms/step - loss: 0.0763
Epoch 175/200
3/3 _____ 0s 13ms/step - loss: 0.0731
Epoch 176/200
3/3 _____ 0s 14ms/step - loss: 0.0752
Epoch 177/200
3/3 _____ 0s 13ms/step - loss: 0.0762
Epoch 178/200
3/3 _____ 0s 13ms/step - loss: 0.0716
Epoch 179/200
3/3 _____ 0s 16ms/step - loss: 0.0717
Epoch 180/200
3/3 _____ 0s 13ms/step - loss: 0.0720
Epoch 181/200
3/3 _____ 0s 13ms/step - loss: 0.0784
Epoch 182/200
3/3 _____ 0s 15ms/step - loss: 0.0721
Epoch 183/200
3/3 _____ 0s 14ms/step - loss: 0.0642
```

```

Epoch 184/200
3/3 _____ 0s 14ms/step - loss: 0.0675
Epoch 185/200
3/3 _____ 0s 14ms/step - loss: 0.0659
Epoch 186/200
3/3 _____ 0s 18ms/step - loss: 0.0698
Epoch 187/200
3/3 _____ 0s 15ms/step - loss: 0.0800
Epoch 188/200
3/3 _____ 0s 16ms/step - loss: 0.0657
Epoch 189/200
3/3 _____ 0s 14ms/step - loss: 0.0644
Epoch 190/200
3/3 _____ 0s 14ms/step - loss: 0.0657
Epoch 191/200
3/3 _____ 0s 16ms/step - loss: 0.0611
Epoch 192/200
3/3 _____ 0s 14ms/step - loss: 0.0678
Epoch 193/200
3/3 _____ 0s 14ms/step - loss: 0.0593
Epoch 194/200
3/3 _____ 0s 15ms/step - loss: 0.0689
Epoch 195/200
3/3 _____ 0s 14ms/step - loss: 0.0642
Epoch 196/200
3/3 _____ 0s 15ms/step - loss: 0.0648
Epoch 197/200
3/3 _____ 0s 15ms/step - loss: 0.0641
Epoch 198/200
3/3 _____ 0s 14ms/step - loss: 0.0588
Epoch 199/200
3/3 _____ 0s 14ms/step - loss: 0.0738
Epoch 200/200
3/3 _____ 0s 13ms/step - loss: 0.0681

```

Generated Text (Transformer):

```

1/1 _____ 1s 846ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 40ms/step
1/1 _____ 0s 37ms/step
1/1 _____ 0s 38ms/step
1/1 _____ 0s 44ms/step

```

deep learning improves sequence modeling performance used used

```

print("\nGenerated Text (Transformer):")
print(generate_text_transformer("long short term", 6))

```

Generated Text (Transformer):

```

1/1 _____ 0s 36ms/step

```

1/1 _____ 0s 37ms/step
1/1 _____ 0s 45ms/step
1/1 _____ 0s 100ms/step
1/1 _____ 0s 63ms/step
1/1 _____ 0s 79ms/step

long short term memory networks solve long dependency problems